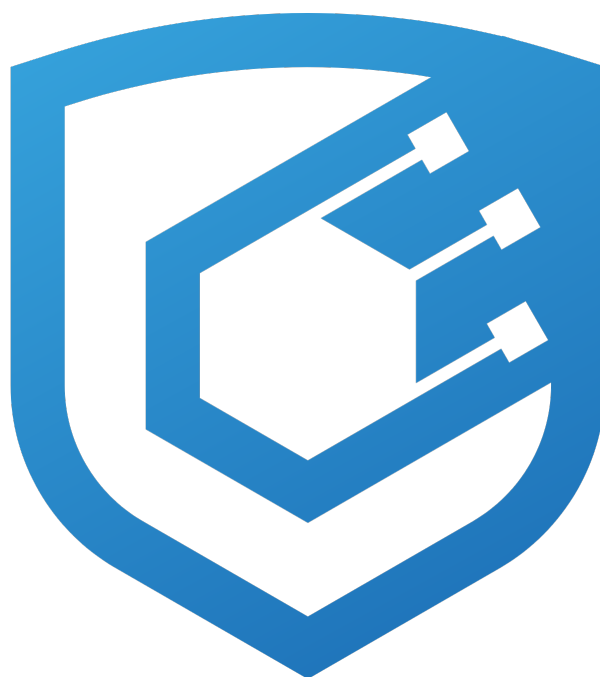




Atum Solana Escrow V2 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Farouk](#)

[JesJupyter](#)

[Ctrus](#)

April 2, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	4
7	Findings	6
7.1	Medium Risk	6
7.1.1	Escrow ATA Balance Mismatch Enables Fee Bypass, Limit Bypass, and Accounting Inconsistency	6
7.2	Low Risk	8
7.2.1	Signatures Lack Cluster and Program Binding, Enabling Cross-Cluster and Cross-Program Replay	8
7.2.2	Missing mint validation in CreateDelegate	8
7.2.3	Fulfillment Proxy Does Not Support Token-2022 TransferHook Extension	9
7.3	Informational	11
7.3.1	Unused Error Variants in Escrow Program	11

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Atum Solana Escrow is a PDA-owned escrow that moves SPL / Token-2022 tokens through a deposit → reserve → release / refund lifecycle under off-chain Ed25519 authorization. A companion Fulfillment Proxy provides direct settler-to-recipient transfers for cross-chain settlement. Both programs share a bitmap-based replay crate with 128-second time-sliced windows.

• State

- ProtocolConfig (Escrow) – [b"protocol_config"]. Fields: default_admin, token_config_authority, treasurer, pauser, paused, default_admin_delay_seconds, pending_default_admin: Option<Pubkey>, pending_default_admin_accept_after_ts: Option<i64>.
- ProtocolConfig (Proxy) – [b"protocol_config"]. Subset: default_admin, pauser, paused, default_admin_delay_seconds, plus pending-admin fields.
- EscrowDelegate – [b"escrow_delegate", authority, mint]. Fields: authority, delegate_signer, max_transfer_size, remaining_capacity, expiration, allowed_mint.
- DepositInfo – [b"deposit", deposit_hash]. Fields: depositor, mint, amount, reserver, releaser, recipient: Option<Pubkey>, deposit_hash: [u8; 32], fee_amount. Owns the escrow ATA.
- MintFeeConfig – [b"mint_fee_config", mint]. Existence = mint is allowlisted. Fields: mint, fee_percentage_6, min_fee_6, max_fee_6.
- ReplayBucket – [b"replay", authority, nonce[4..8]]. Fields: bitmaps: [[u64; 4]; 2] — two parity bitmaps; word 3 packs a sentinel bit (63), a 20-bit epoch (bits 43–62), and slots 192–234. 235 usable slots per parity.

• External dependencies

- **Ed25519 program** plus instructions sysvar to validate the immediately preceding signature instruction.
- **SPL Token / Token-2022** via token_interface and **Associated Token** for ATA creation, transfer_checked, approve_checked, revoke, and close_account. Mints with TransferFeeConfig are rejected; post-transfer hooks forwarded via remaining_accounts in escrow only.

- **Fulfillment Proxy** depends on Token program only (does not forward `remaining_accounts`).
- **Core instructions (Escrow)**
 - **create_delegate** – creates `EscrowDelegate`, approves SPL delegation on the authority ATA for `remaining_capacity`, pins `allowed_mint`, sets `max_transfer_size` and `expiration`.
 - **renounce_delegate** – callable by `delegate_signer`. Sets `remaining_capacity` = 0 and `expiration` = `now`. Does not revoke SPL approval.
 - **revoke_delegate** – callable by authority. Revokes SPL approval on authority ATA, closes `EscrowDelegate`, returns rent to payer.
 - **deposit** – creates `DepositInfo` and escrow ATA, checks replay window, capacity and size limits, `allowed_mint`, verifies `Ed25519` delegate signature on `keccak256("deposit" || payment_id || depositor || mint || amount || reserver || releaser || nonce || issued_at || deadline)`, computes and snapshots fee, transfers from authority ATA to escrow ATA via PDA signing, verifies exact amount received.
 - **reserve** – sets `recipient` (one-time). Verifies `reserver` signature on `keccak256("reserve" || recipient || deposit_hash)`.
 - **release** – verifies `releaser` signature on `keccak256("release" || deposit_hash)`, transfers `net = escrow_ata.amount - fee_amount` to `recipient ATA`, fee to `treasury ATA`, closes escrow ATA and `DepositInfo`.
 - **refund** – verifies `releaser` signature on `keccak256("refund" || deposit_hash)`, transfers entire escrow ATA balance to `depositor ATA`, closes escrow ATA and `DepositInfo`.
- **Core instructions (Fulfillment Proxy)**
 - **fulfill** – settler-signed token transfer, no replay protection. Emits `Fulfilled`.
 - **strict_fulfill** – settler-signed token transfer with replay-bucket check (deadline, 128-second window, duplicate nonce). Emits `StrictFulfilled`.
- **Admin instructions (both programs)**
 - **initialize_protocol** – callable by program upgrade authority only (V3/V4 loader). Creates `ProtocolConfig` with all role keys. Uses `init_if_needed` with a manual initialization guard.
 - **pause / unpause** – pauser toggles `paused` flag; all user-facing instructions check it.
 - **set_pauser, set_treasurer, set_token_config_authority** – `default_admin` rotates role keys (treasurer and `token_config_authority` are escrow-only).
 - **begin_default_admin_transfer / accept_default_admin_transfer** – two-step admin transfer with configurable `default_admin_delay_seconds`.
 - **allow_mint / disallow_mint / set_mint_fee_config / clear_mint_fee_config** – `token_config_authority` manages mint allowlist and per-mint fee parameters (escrow only).
- **Fee computation**
 - `fee = floor(amount × fee_percentage_6 / 100_000_000)`, clamped to `[min_fee, max_fee]` converted from 6-decimal to atomic units via mint decimals. Atomic fallback: if `percentage > 0` but computed fee rounds to 0, fee is set to 1. Fee is snapshotted in `DepositInfo.fee_amount` at deposit time and applied at release.

5 Audit Scope

The audit scope was limited to:

```
programs/escrow/src/instructions/delegate.rs
programs/escrow/src/instructions/escrow.rs
programs/escrow/src/instructions/mint_config.rs
```

```
programs/escrow/src/instructions/protocol.rs
programs/escrow/src/utils/fees.rs
programs/escrow/src/utils/replay.rs
programs/escrow/src/utils/signatures.rs
programs/escrow/src/lib.rs
programs/fulfillment_proxy/src/instructions/fulfillment.rs
programs/fulfillment_proxy/src/instructions/protocol.rs
programs/fulfillment_proxy/src/lib.rs
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Solana Escrow V2](#) smart contracts provided by [Atum](#). In this period, a total of 5 issues were found.

Atum Solana Escrow is a two-program payment settlement system providing PDA-controlled token escrow with Ed25519-authorized lifecycle transitions and a companion fulfillment proxy for cross-chain settlement. The security model relies on correct signature verification, role-based access enforcement, deterministic fee accounting, and bitmap-based replay protection across both programs.

Our review focused on the escrow deposit-reserve-release-refund lifecycle, Ed25519 signature verification, RBAC and admin transfer mechanics, fee computation and split logic, Token-2022 compatibility handling, replay protection correctness, and the fulfillment proxy's transfer flows. The audit identified 5 findings: 1 Medium, 3 Low, and 1 Informational. No critical vulnerabilities were discovered.

The medium severity finding relates to the escrow release instruction using the live ATA balance rather than the stored deposit amount as the gross for fee splitting, which allows depositors to dilute effective fee rates and bypass transfer size limits by sending additional tokens to the escrow account before release. The low severity findings address the absence of cluster and program ID binding in signed messages (enabling theoretical cross-environment replay), missing mint allowlist validation during delegate creation (causing wasted rent for disallowed mints), and the fulfillment proxy's omission of remaining accounts forwarding in token transfer CPIs (preventing TransferHook-enabled Token-2022 tokens from being used). The informational item identifies unused error variants in the escrow program.

Overall, the codebase demonstrates a well-structured security posture with thorough role separation, defense-in-depth fee validation, and sound escrow lifecycle design.

Summary

Project Name	Solana Escrow V2
Repository	solana-escrow
Commit	7436f2ae20d7...
Audit Timeline	Feb 20th - Feb 26th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	1
Low Risk	3
Informational	1
Gas Optimizations	0
Total Issues	5

Summary of Findings

[M-1] Escrow ATA Balance Mismatch Enables Fee Bypass, Limit Bypass, and Accounting Inconsistency	Resolved
[L-1] Signatures Lack Cluster and Program Binding, Enabling Cross-Cluster and Cross-Program Replay	Resolved
[L-2] Missing mint validation in <code>CreateDelegate</code>	Resolved
[L-3] <code>Fulfillment Proxy</code> Does Not Support Token-2022 <code>TransferHook</code> Extension	Resolved
[I-1] Unused Error Variants in Escrow Program	Resolved

7 Findings

7.1 Medium Risk

7.1.1 Escrow ATA Balance Mismatch Enables Fee Bypass, Limit Bypass, and Accounting Inconsistency

Description: Background: This behavior stems from the fix for **C-01** (donation-attack DOS on `close_account`). At the time of that fix, fees were not yet implemented, so using `escrow_ata.amount` as the gross and draining the entire balance was fully correct in the previous version of the codebase. The issue only surfaced after fees were introduced, which changed the assumptions under which the original fix was designed: the fee split now relies on a snapshot of `deposit.fee_amount` while the gross is dynamic, creating a mismatch.

The release instruction uses `ctx.accounts.escrow_ata.amount` (current ATA balance) as `gross` to prevent donation-attack DOS on `close_account` (C-01 fix in previous audit).

```
// 3. Fee split calculation
let gross = ctx.accounts.escrow_ata.amount;
let fee = deposit.fee_amount;

require!(gross >= fee, ErrorCode::InsufficientEscrowBalanceForFee);

let net = gross.checked_sub(fee).unwrap();
```

At deposit time, the fee is computed and stored in `deposit.fee_amount`:

```
let fee_amount = compute_fee(
    amount,
    mint_decimals,
    mint_fee_config.fee_percentage_6,
    mint_fee_config.min_fee_6,
    mint_fee_config.max_fee_6,
)?;
```

At release time, `gross` is taken from the current escrow ATA balance, while `fee` remains the snapshot value:

```
// 3. Fee split calculation
let gross = ctx.accounts.escrow_ata.amount;
let fee = deposit.fee_amount;

require!(gross >= fee, ErrorCode::InsufficientEscrowBalanceForFee);

let net = gross.checked_sub(fee).unwrap();
```

However, there is no guarantee that the escrow ATA balance equals the original `deposit.amount`. Because SPL token accounts accept incoming transfers from anyone, the balance can be altered before release.

- Attack Scenario 1: Same User (Depositor) Donation

1. Depositor creates a deposit of 100 tokens. Fee is computed as 5 (5%), stored in `deposit.fee_amount`. `remaining_capacity` and `max_transfer_size` are enforced (e.g., max 100).
2. Before release, the depositor sends 50 additional tokens directly to the escrow ATA via a normal SPL transfer.
3. On release: `gross = 150`, `fee = 5`, `net = 145`.
4. Recipient receives 145; treasury receives 5.

Leading to:

- **Fee bypass:** Effective fee rate = $5/150$ 3.33% instead of 5%. Protocol revenue is diluted.
- **Limit bypass:** `max_transfer_size` and `remaining_capacity` were enforced only at deposit. The effective payment is 150, exceeding the intended limits.

- **Payment flexibility:** The depositor can change the payment amount at any time before release by donating more tokens.
 - Attack Scenario 2: Third-Party Donation (Grief / Inconsistency)
1. User A deposits 100 tokens for a recipient. Off-chain payment system expects recipient to receive `net = 95` (`100 - 5 fee`).
 2. User B (or any address) sends 1 token to the escrow ATA.
 3. On release: `gross = 101`, `fee = 5`, `net = 96`.
 4. Recipient receives 96 instead of the expected 95.

Leading to:

- **Accounting inconsistency:** Off-chain systems that track expected amounts will mismatch on-chain results. Integrations that assume `net == deposit.amount - deposit.fee_amount` will break.

Impact:

1. **Protocol revenue loss:** Depositors can reduce effective fee rate by donating extra tokens.
2. **Invariant violation:** `max_transfer_size` and `remaining_capacity` can be circumvented.
3. **Integration risk:** Off-chain systems that assume deterministic `net` from `deposit.amount` and `deposit.fee_amount` will see incorrect amounts.
4. **Unpredictable payments:** Payment amount can be changed by anyone before release, undermining predictability for recipients and integrators.

Recommended Mitigation:

- Store the `fee_rate` and apply it during actual release on `gross`.
- And add intended `deposit.amount` into the Released event.

Atum: Fixed in [b4c128e](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 Signatures Lack Cluster and Program Binding, Enabling Cross-Cluster and Cross-Program Replay

Description: `compute_deposit_hash` builds the signed message without cluster or program context:

```
// 4. Signature Verification
let message = compute_deposit_hash(
  payment_id,
  delegate.authority,
  delegate.allowed_mint,
  amount,
  reserver,
  releaser,
  nonce,
  issued_at,
  deadline,
);
```

The deposit (and derived reserve/release/refund) signatures do not include cluster-specific data or the program ID. This allows:

- Cross-cluster replay: A signature created for Devnet/Testnet to be replayed on Mainnet.
 1. User signs a deposit on Devnet for testing.
 2. Attacker captures the Ed25519 instruction and signature.
 3. Attacker submits the same instruction on Mainnet.
 4. Mainnet replay bucket has not seen this nonce, so the replay check passes.
 5. If delegate, ATA, and other accounts exist on Mainnet, the deposit succeeds and real funds are moved.
- Cross-program replay: A signature to be replayed on a forked program instance with a different program ID. If the user happens to interact with both programs, the signature could be replayed.

Impact: Cross-cluster: Users testing on Devnet/Testnet can have their signatures replayed on Mainnet, leading to unintended deposits and fund movement. Cross-program: Signatures can be replayed on fork instances

Recommended Mitigation:

1. Include `program_id` in the signed message
2. Add a `cluster_id` (or similar) to the signed message.

Atum: Fixed in [b4c128e](#).

Cyfrin: Verified.

7.2.2 Missing mint validation in `CreateDelegate`

Description: The `CreateDelegate` instruction creates an `EscrowDelegate` account and approves token delegation without verifying that the mint is on the protocol's allowlist (i.e., that a `MintFeeConfig` PDA exists for the mint).

In contrast, the `deposit` function in `escrow.rs` does validate the mint allowlist:

```
// In deposit handler:
let mint_fee_config_info = &ctx.accounts.mint_fee_config;
require!(
  !mint_fee_config_info.data_is_empty(),
  ErrorCode::MintNotAllowed
);
```

This means users can create delegate accounts for mints that are not allowed by the protocol. When they later attempt to deposit using this delegate, the transaction will fail with `MintNotAllowed`, but the rent paid for the `EscrowDelegate` account (128 bytes) has already been spent.

Impact: Users pay rent to create accounts that are immediately unusable if the mint is not allowlisted.

Recommended Mitigation: Add a `MintFeeConfig` account check to the `CreateDelegate` instruction to ensure the mint is on the allowlist before creating the delegate

Atum: Fixed in [b4c128e](#).

Cyfrin: Verified.

7.2.3 Fulfillment Proxy Does Not Support Token-2022 TransferHook Extension

Description: The escrow program correctly forwards `remaining_accounts` in all token transfer CPI calls:

```
token::transfer_checked(
  CpiContext::new_with_signer(
    ctx.accounts.token_program.to_account_info(),
    TransferChecked {
      from: ctx.accounts.authority_ata.to_account_info(),
      to: ctx.accounts.escrow_ata.to_account_info(),
      authority: ctx.accounts.escrow_delegate.to_account_info(),
      mint: ctx.accounts.mint.to_account_info(),
    },
    signer_seeds,
  )
  .with_remaining_accounts(ctx.remaining_accounts.to_vec()),
  amount,
  decimals,
)?;
```

The same pattern is used in `releaseand` and `refund`.

The `fulfillment_proxy` program performs token transfers **without** forwarding `remaining_accounts`:

```
token::transfer_checked(
  CpiContext::new(
    ctx.accounts.token_program.to_account_info(),
    TransferChecked {
      from: ctx.accounts.from_ata.to_account_info(),
      to: ctx.accounts.to_ata.to_account_info(),
      authority: ctx.accounts.settler.to_account_info(),
      mint: ctx.accounts.mint.to_account_info(),
    },
  ),
  amount,
  ctx.accounts.mint.decimals,
)?;
```

The `strict_fulfill` instruction (lines 61–74) has the same omission. Even if the client passes `TransferHook` accounts as remaining accounts, they are ignored because the program does not call `.with_remaining_accounts(ctx.remaining_accounts.to_vec())`.

Impact:

- Users cannot fulfill payments using Token-2022 tokens with `TransferHook`. The transfer CPI will fail.
- Escrow supports `TransferHook` but `fulfillment_proxy` does not.

Recommended Mitigation: Add `.with_remaining_accounts(ctx.remaining_accounts.to_vec())` to `transfer_checked` CPI calls.

Atum: Fixed in [b4c128e](#).

Cyfrin: Verified.

7.3 Informational

7.3.1 Unused Error Variants in Escrow Program

Description: Several error variants are defined in the escrow program's `ErrorCode` enum but are never used in the codebase. For example:

```
#[msg("Deposit with this ID already exists")]
DepositAlreadyExists,
#[msg("Deposit not found")]
DepositNotFound,
#[msg("Deposit has already been released")]
DepositAlreadyReleased,
#[msg("Deposit has already been refunded")]
DepositAlreadyRefunded,
```

The program never reaches a point where it could return these custom errors.

Impact: Unused enum variants add noise and can suggest to future readers that explicit checks exist when they do not. Removing them (or wiring them to explicit checks for clearer client errors) would simplify the codebase.

Recommended Mitigation: Remove the four unused error variants. If the team wants more descriptive client-facing errors, add explicit `require!` checks that return these errors.

Atum: Fixed in [b4c128e](#).

Cyfrin: Verified.